

```
pip install git+https://github.com/mimoralea/gym-walk#egg=gym-walk
```

```
Collecting gym-walk
  Cloning https://github.com/mimoralea/gym-walk to /tmp/pip-install-n_c1b4xq/gym-walk_90eeabc8bb6d49a7a48fe62113f503bf
  Running command git clone --filter=blob:none --quiet https://github.com/mimoralea/gym-walk /tmp/pip-install-n_c1b4xq/gym-walk_90eeabc8bb6d49a7a48fe62113f503bf
  Resolved https://github.com/mimoralea/gym-walk to commit b915b94cf2ad16f8833a1ad92ea94e88159279f5
  Preparing metadata (setup.py) ... done
Requirement already satisfied: gym in /usr/local/lib/python3.11/dist-packages (from gym-walk) (0.25.2)
Requirement already satisfied: numpy>=1.18.0 in /usr/local/lib/python3.11/dist-packages (from gym->gym-walk) (2.0.2)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from gym->gym-walk) (3.1.1)
Requirement already satisfied: gym_notices>=0.0.4 in /usr/local/lib/python3.11/dist-packages (from gym->gym-walk) (0.0.8)
```

```
import warnings ; warnings.filterwarnings('ignore')

import gym, gym_walk
import numpy as np

import random
import warnings

warnings.filterwarnings('ignore', category=DeprecationWarning)
np.set_printoptions(suppress=True)
random.seed(123); np.random.seed(123)
```

```
def print_policy(pi, P, action_symbols=('<', 'v', '>', '^'), n_cols=4, title='Policy:'):
    print(title)
    arrs = {k:v for k,v in enumerate(action_symbols)}
    for s in range(len(P)):
        a = pi(s)
        print("| ", end="")
        if np.all([done for action in P[s].values() for _, _, done in action]):
            print("".rjust(9), end=" ")
        else:
            print(str(s).zfill(2), arrs[a].rjust(6), end=" ")
        if (s + 1) % n_cols == 0: print("|")
```

```
def print_state_value_function(V, P, n_cols=4, prec=3, title='State-value function:'):
    print(title)
    for s in range(len(P)):
        v = V[s]
        print("| ", end="")
        if np.all([done for action in P[s].values() for _, _, done in action]):
            print("".rjust(9), end=" ")
        else:
            print(str(s).zfill(2), '{}'.format(np.round(v, prec)).rjust(6), end=" ")
        if (s + 1) % n_cols == 0: print("|")
```

```
def probability_success(env, pi, goal_state, n_episodes=100, max_steps=200):
    random.seed(123); np.random.seed(123) ; env.seed(123)
    results = []
    for _ in range(n_episodes):
        state, done, steps = env.reset(), False, 0
        while not done and steps < max_steps:
            state, _, done, h = env.step(pi(state))
            steps += 1
        results.append(state == goal_state)
    return np.sum(results)/len(results)
```

```
def mean_return(env, pi, n_episodes=100, max_steps=200):
    random.seed(123); np.random.seed(123) ; env.seed(123)
    results = []
    for _ in range(n_episodes):
        state, done, steps = env.reset(), False, 0
        results.append(0.0)
        while not done and steps < max_steps:
            state, reward, done, _ = env.step(pi(state))
            results[-1] += reward
            steps += 1
    return np.mean(results)
```

```
desc=['FFSH', 'HFFH', 'HFGH', 'FHFF']
env = gym.make('FrozenLake-v1',desc=desc)
init_state = env.reset()
goal_state = 10
P = env.env.P
```

```
def value_iteration(P, gamma=1.0, theta=1e-10):
    V = np.zeros(len(P), dtype=np.float64)
    while True:
        Q=np.zeros((len(P),len(P[0])),dtype=np.float64)
        for s in range(len(P)):
            for a in range(len(P[s])):
                for prob,next_state,reward,done in P[s][a]:
                    Q[s][a]+=prob*(reward+gamma*V[next_state]*(not done))
        if np.max(np.abs(V-np.max(Q,axis=1)))<theta:
            break
        V=np.max(Q,axis=1)
    pi=lambda s:{s:a for s,a in enumerate(np.argmax(Q,axis=1))}[s]
    return V, pi
```

```
V_best_v, pi_best_v = value_iteration(P, gamma=0.99)
```

```
print("Name:NARESH.R          Register Number:212223240104")
print('Optimal policy and state-value function (VI):')
print_policy(pi_best_v, P)
```

```
↗ Name:NARESH.R          Register Number:212223240104
Optimal policy and state-value function (VI):
Policy:
| 00      ^ | 01      > | 02      < |          |
|          | 05      > | 06      < |          |
|          | 09      > |          |          |
| 12      < |          | 14      > | 15      v |
```

```
print('Reaches goal {:.2f}%. Obtains an average undiscounted return of {:.4f}.'.format(
    probability_success(env, pi_best_v, goal_state=goal_state)*100,
    mean_return(env, pi_best_v)))
```

```
↗ Reaches goal 83.00%. Obtains an average undiscounted return of 0.8300.
```

```
print_state_value_function(V_best_v, P, prec=4)
```

```
↗ State-value function:
| 00 0.677 | 01 0.6975 | 02 0.7379 |          |
|          | 05 0.6783 | 06 0.8007 |          |
|          | 09 0.5572 |          |          |
| 12 0.0   |          | 14 0.9532 | 15 0.9251 |
```

Start coding or [generate](#) with AI.